

TASK 2:

1. `git log`

What it does:

Shows the commit history in your current branch. You can see the order of commits, their messages, author, and commit ID.

Example usage:

```
bash
Copy code
git log
```

Shows a detailed list of commits with dates, authors, and messages.

Why it's useful:

Helps track changes, find when bugs were introduced, or understand project history.

2. `git remote`

What it does:

Manages connections to remote repositories, like those on GitHub.

Example usage:

```
bash
Copy code
git remote -v
```

Lists the URLs of remotes, usually `origin` for your GitHub repo.

You can also:

- Add a remote with `git remote add <name> <url>`
 - Remove or rename remotes.
-

3. `git checkout`

What it does:

Switches branches or restores files from a commit.

Example usage:

bash

Copy code

```
git checkout feature-branch
```

Switches to `feature-branch`.

Or create and switch to a new branch:

bash

Copy code

```
git checkout -b new-branch
```

Why it's useful:

Allows you to work on different features independently.

4. `git revert`

What it does:

Creates a new commit that undoes changes made by a previous commit, without deleting history.

Example usage:

bash

Copy code

```
git revert <commit-hash>
```

Why it's useful:

Safely undo changes while preserving the commit history.

5. `git add`

What it does:

Stages changes (new or modified files) to be included in the next commit.

Example usage:

bash

Copy code

```
git add file.txt
```

Or stage everything:

bash

Copy code

```
git add .
```

Why it's useful:

Prepares changes for commit, letting you select which files to commit.

6. git commit

What it does:

Records staged changes as a new commit with a message describing the change.

Example usage:

bash

Copy code

```
git commit -m "Fix typo in README"
```

Why it's useful:

Saves your progress and documents what you've done.

7. git diff

What it does:

Shows the difference between files or commits.

Example usage:

bash

Copy code

```
git diff
```

Shows unstaged changes in your working directory.

To see staged changes:

bash

Copy code

```
git diff --staged
```

Why it's useful:

Helps review what changes have been made before committing.

8. git fetch

What it does:

Downloads changes from the remote repository but does not merge them into your branch.

Example usage:

bash

Copy code

```
git fetch origin
```

Why it's useful:

Allows you to see new remote commits without affecting your current work.

9. git push

What it does:

Uploads your commits from the local repository to the remote repository.

Example usage:

bash

Copy code

```
git push origin feature-branch
```

Why it's useful:

Shares your changes with others and updates remote branches.

10. `git pull`

What it does:

Fetches changes from a remote repo and merges them into your current branch.

Example usage:

bash

Copy code

```
git pull origin main
```

Why it's useful:

Keeps your local repo up to date with remote changes.